Python Cheatsheet

Essentials of Data Science in Python · Session 6 · Causal Inference

Thierry Warin, PhD · HEC Montréal

Everything you need to type, in one place. The first four sections are the same every week; the last one is every name from sessions 1 to 6.

Starting Python in VS Codium

Open the course folder once — **File > Open Folder**, then pick the repository. Everything below assumes you are inside it, and in **the VS Codium terminal** (**View > Terminal**, or `Ctrl` + `` ` ``) rather than the macOS Terminal or PowerShell.

Create the environment the first time only:

```
python3 -m venv .venv      # macOS
python3 -m venv .venv      # Linux
py -m venv .venv           # Windows (PowerShell)
```

Then activate it — **every time you open a new terminal**:

```
source .venv/bin/activate  # macOS
source .venv/bin/activate  # Linux
.venv\Scripts\activate     # Windows (PowerShell)
```

You will see `(.venv)` at the start of the prompt. If you do not, nothing below will find pandas. Install the packages once, with the environment active, and check the course data is intact:

```
pip install -r requirements.txt
python scripts/verify_puzzles.py    # 39 lines of [  ok  ], in about twenty seconds
```

Finally tell the editor which Python to use: `Ctrl`/`Cmd` + `Shift` + `P`, type **Python: Select Interpreter**, and choose the one inside `.venv`.

A new file, and running it

File > New File, then save it with a `.py` ending — `session06.py`, say. Write your code, save it, and in the terminal:

```
python session06.py     # macOS, Linux and Windows alike, inside .venv
```

Inside the environment the command is `python` on all three platforms. Outside it, macOS and Linux need `python3` and Windows needs `py`, which is one more reason to activate first. For trying one line at a time, type `python` on its own to get the `>>>` prompt, and `exit()` to leave.

The four things you always do

```
import numpy as np
import pandas as pd
import essentials as ess

# 1 - load a course dataset, or build a frame by hand
d = ess.load("cat_cafe")                # generated locally; no network needed
ess.catalog()                              # the other five, and their puzzles
df = pd.DataFrame({"country": ["FR", "DE"], "gdp_pc_eur": [37_000, 46_000]})

# 2 - look at what is in it
d.head()           # the first five rows
```

```
d.shape            # (rows, columns)
d.dtypes           # what type each column holds
d.info()           # types, and how many values are missing
d.describe()       # count, mean, sd and quartiles, per numeric column

# 3 - read a file in
df = pd.read_csv("data.csv")
df = pd.read_excel("data.xlsx")             # needs openpyxl - see below
df = pd.read_parquet("data/coasters.parquet")

# 4 - write a CSV out
d.to_csv("out.csv", index=False)            # index=False, or you get a stray column
```

`import essentials as ess` works from the repository root. From a subfolder — a starter script in `06-causal-inference-did/02-practice/starter/`, say — point Python back at the root first, which is what the two lines at the top of every starter do:

```
import sys
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parents[3]))
import essentials as ess
```

Installing a package you do not have

Everything this course needs is already in `requirements.txt`. When you meet a `ModuleNotFoundError` anyway, or a line above says a reader needs something extra, install it **into the environment** — never outside it:

```
pip install openpyxl                # one package, by name
pip install -r requirements.txt     # or the whole course list again
```

Two rules, and both are about where it lands:

- Your prompt must show `(.venv)` **first**. `pip -V` prints the path it is about to install into; that path has to contain `.venv`.
- Type `pip`, not `pip3`, and never `sudo`. Inside an activated environment `pip` is already the environment's own.

Then check it arrived:

```
pip show openpyxl                   # name, version and location - or nothing at all
python -c "import openpyxl; print(openpyxl.__version__)"
```

What you have learned so far

Sessions 1 to 6, in the order you met them. Every line below has run on the course data.

Session 1 · Regression

```
# -- Load and shape -----------------------------------------------
import essentials as ess
d = ess.load("coasters")          # generated locally, no network
ess.catalog()                     # every dataset, and its puzzle
n_rows, n_cols = d.shape
d[["coaster", "drop_m", "speed_kmh"]].head()
g = d[d.propulsion == "gravity"].copy()
# .copy() before you modify a subset, or pandas warns and you guess
g["log_speed"] = np.log(g.speed_kmh)      # a power law becomes a line
c = np.exp(power.params["Intercept"])     # and back again
bands = pd.cut(g.drop_m, [0, 25, 45, 65, 85, 105, 200])
(g.assign(resid=lin.resid)
```

```
  .groupby(bands, observed=True)["resid"].agg(["mean", "count"]))
# the sign of the mean residual, band by band, is the whole diagnosis

# -- Describe ------------------------------------------------------
d.describe()                      # count, mean, sd, quartiles
d.propulsion.value_counts()       # how many of each

# -- Fit -----------------------------------------------------------
import statsmodels.formula.api as smf
lin = smf.ols("speed_kmh ~ drop_m", data=g).fit()
# read the ~ as "explained by"; the intercept is added for you
fit = model.fit()                 # .fit() does the arithmetic

# -- Read the fit --------------------------------------------------
lin.params["drop_m"], lin.bse["drop_m"]   # estimate, and its wobble
lin.tvalues["drop_m"], lin.pvalues["drop_m"]
lo, hi = lin.conf_int().loc["drop_m"]     # 95% by default
lin.rsquared, lin.rsquared_adj    # the second penalises extra columns
lin.resid, lin.fittedvalues       # what is left, and what was predicted
print(lin.summary().tables[1])    # the coefficient table on its own

# -- Diagnose ------------------------------------------------------
inf = lin.get_influence()         # the whole diagnostic bundle
h = inf.hat_matrix_diag           # leverage; it sums to p, always
(h > 2 * 2 / len(d)).sum()        # the 2p/n screen
cooks = inf.cooks_distance[0]     # leverage x residual; screen at 4/n

# -- Plot ----------------------------------------------------------
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7.2, 3.2))
ax.scatter(g.drop_m, g.speed_kmh, s=20, alpha=0.8)
ax.set_xlabel("Drop (m)"); ax.set_ylabel("Top speed (km/h)")
# say the units, every time
ax.axhline(0, lw=1)               # the line you are judging against
ax.axhline(4 / len(d), ls="--", lw=1)     # the Cook's screen
ax.vlines(range(len(cooks)), 0, cooks, lw=1.4)   # one spike per row
plt.show()                        # last line of every figure
```

Session 2 · Advanced Regression

```
# -- Load and shape ------------------------------------------------
d.baker.nunique()                 # eight bakers
Z = full.predict(grid).to_numpy().reshape(HH.shape)
HH.ravel()                        # a grid, flattened into a column

# -- Fit -----------------------------------------------------------
quad = smf.ols("rise_mm ~ hydration_pct + I(hydration_pct**2)",
               data=d).fit()
# I() means: treat what is inside as arithmetic, not as formula syntax
fe = smf.ols("rise_mm ~ hydration_pct + oven_c + C(baker)",
             data=d).fit()
# one dummy per baker, one held back as the reference
"rise_mm ~ hydration_pct + oven_c + hydration_pct:oven_c"
# the slope on water now depends on the oven, and vice versa
```

```
# -- Read the fit ------------------------------------------------
row = pd.DataFrame([{"hydration_pct": 78, "oven_c": 228, "baker": "D"}])
fit.predict(row)                  # a fitted value at a point you choose
pred = fit.get_prediction(grid).summary_frame()
pred["mean"], pred["mean_ci_lower"], pred["mean_ci_upper"]
fit.aic, fit.bic                  # lower is better, and only relatively

# -- Choose and validate -----------------------------------------
f_stat, p_val, _ = small.compare_f_test(big)
# only for nested models: small must be big with terms removed
from scipy.optimize import minimize
r = minimize(neg_predicted, [75, 225], bounds=[(55, 95), (190, 260)])
# find the peak of a fitted surface numerically, inside the design
H_TRUE, T_TRUE = ess.sourdough_optimum()  # the planted answer, to score against

# -- Plot --------------------------------------------------------
hs = np.linspace(58, 94, 60); ts = np.linspace(195, 258, 60)
HH, TT = np.meshgrid(hs, ts)
cs = ax.contour(HH, TT, Z, levels=12, linewidths=0.7)
ax.clabel(cs, inline=True, fontsize=6, fmt="%.0f")
ax.contourf(HH, TT, Z, levels=24, cmap="YlOrBr", alpha=0.45)
ax.fill_between(hs, pred["mean_ci_lower"], pred["mean_ci_upper"],
                alpha=0.18)       # the interval, as a band
ax.annotate("planted optimum", (H_TRUE, T_TRUE), (H_TRUE - 17, T_TRUE + 9))
```

Session 3 · Logistic Regression

```
# -- Load and shape ----------------------------------------------
(p_hat > t).astype(int)           # a probability becomes a call
sub = d.dropna(subset=["toxicity"]).copy()
# subset= matters: without it a row missing ANY column goes
X = pd.get_dummies(sub[["odor_index", "spore_print"]],
                   columns=["spore_print"], drop_first=True, dtype=float)
sub.toxicity.cat.codes            # an ordered category as 0, 1, 2, 3
counts.reindex(["edible", "upset stomach", "hospital", "fatal"])
# print an ordered category in its own order, not alphabetically

# -- Fit ---------------------------------------------------------
fit = smf.logit("poisonous ~ odor_index + volva_present",
                data=d).fit(disp=0)
"... + C(spore_print, Treatment('brown'))"
# choose the reference level yourself, rather than taking the first
from statsmodels.miscmodels.ordinal_model import OrderedModel
ord_fit = OrderedModel(sub.toxicity.cat.codes, X,
                       distr="logit").fit(method="bfgs", disp=0)

# -- Read the fit ------------------------------------------------
np.exp(fit.params)                # a log-odds becomes an odds ratio
# an odds ratio is not a probability: what it does to p depends on p
p_hat = fit.predict(d)            # probabilities, not classes
fit.summary2().tables[1].loc[rows, ["Coef.", "Std.Err.", "z", "P>|z|"]]
lr = 2 * (fit.llf - reduced.llf)  # the likelihood-ratio statistic
fit.prsquared                     # McFadden's pseudo R2
```

```
from scipy import stats
stats.chi2.sf(lr, df)             # its p-value

# -- Choose and validate ----------------------------------------
from sklearn.metrics import confusion_matrix
tn, fp, fn, tp = confusion_matrix(d.poisonous, calls).ravel()
# fn is the one that matters here: a poisonous mushroom called edible
from sklearn.metrics import roc_auc_score, roc_curve
roc_auc_score(d.poisonous, p_hat)
fpr, tpr, thr = roc_curve(d.poisonous, p_hat)
C_FN, C_FP = 50, 1                # a hospital visit against a lost meal
t_star = C_FP / (C_FP + C_FN)     # = 1/51, not 0.5
loss = C_FN * fn + C_FP * fp      # what you are actually minimising
```

Session 4 · Ridge, Lasso, Elastic Net

```
# -- Load and shape ---------------------------------------------
X = r.drop(columns=["bake_id", "taste_score"])
y = r.taste_score.to_numpy()
from sklearn.preprocessing import StandardScaler
Xs = StandardScaler().fit_transform(X.values)
# a penalty on raw columns penalises whatever is measured in small units
from sklearn.model_selection import train_test_split
Xtr, Xte, ytr, yte = train_test_split(Xs, y, test_size=0.35,
                                      random_state=7)

# -- Describe ---------------------------------------------------
X.baking_soda.corr(X.baking_powder)      # 0.96: the lasso will pick one

# -- Fit --------------------------------------------------------
from sklearn.linear_model import LinearRegression
ols = LinearRegression().fit(Xtr, ytr)   # p > n: it fits perfectly, and lies
from sklearn.linear_model import Ridge, Lasso, ElasticNet
Ridge(alpha=lam)                  # shrinks everything, zeroes nothing
Lasso(alpha=lam)                  # shrinks, and zeroes
ElasticNet(alpha=lam, l1_ratio=0.95)     # both, and keeps pairs together
from sklearn.linear_model import RidgeCV, LassoCV, ElasticNetCV
lasso = LassoCV(cv=kf, random_state=0, max_iter=200_000).fit(Xs, y)
lasso.alpha_                      # the lambda cross-validation chose
from sklearn.linear_model import lasso_path
lam, coefs, _ = lasso_path(Xs, y, n_alphas=120, eps=1e-3)

# -- Read the fit -----------------------------------------------
coef = pd.Series(model.coef_, index=X.columns)
sel = coef[coef.abs() > 1e-6]     # what survived

# -- Choose and validate ----------------------------------------
from sklearn.model_selection import KFold, cross_val_score
kf = KFold(10, shuffle=True, random_state=0)
-cross_val_score(model, Xs, y, cv=kf,
                 scoring="neg_mean_squared_error").mean()
from sklearn.metrics import mean_squared_error
mean_squared_error(yte, ols.predict(Xte))   # against y.var(), not against 0
```

```
# -- Plot ------------------------------------------------------------
alphas = np.logspace(-2, 4, 25)   # a penalty grid is log-spaced
ax.plot(np.log10(lam), coefs[j])  # so plot it on a log axis
```

Session 5 · Classification

```
# -- Load and shape --------------------------------------------------
post = ess.bestiary_posterior(d.wing_span_cm, d.tail_length_cm)
SPECIES = ess.bestiary_species()
# the true class probabilities: the Bayes floor no model can beat
sc = StandardScaler().fit(Xtr)    # fit on TRAIN
A, B = sc.transform(Xtr), sc.transform(Xte)   # transform both
# fitting the scaler on everything leaks the test set into the model
train_test_split(X, y, test_size=0.3, random_state=1, stratify=y)
# without it, a 2% species can miss the test set entirely
mesh = np.column_stack([GX.ravel(), GT.ravel()])   # a grid to predict on
pd.Categorical(pred, categories=SPECIES).codes     # labels -> integers

# -- Fit -------------------------------------------------------------
from sklearn.neighbors import KNeighborsClassifier
m = KNeighborsClassifier(45).fit(A, ytr)
# k is the whole model: k=1 memorises, k=n predicts the majority
from sklearn.linear_model import LogisticRegression
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.tree import DecisionTreeClassifier
LogisticRegression(max_iter=5000)        # a straight boundary
QuadraticDiscriminantAnalysis()          # a curved one
DecisionTreeClassifier(max_depth=3, random_state=0)

# -- Read the fit ----------------------------------------------------
m.score(A, ytr), m.score(B, yte)  # train and test, always both
pred = m.predict(B)

# -- Choose and validate ---------------------------------------------
from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(KNeighborsClassifier(),
                    {"n_neighbors": list(range(1, 151, 2))},
                    cv=KFold(10, shuffle=True, random_state=0)).fit(A, ytr)
grid.best_params_, grid.best_score_
from sklearn.metrics import accuracy_score, balanced_accuracy_score
accuracy_score(yte, pred)         # can be high while a class is invisible
balanced_accuracy_score(yte, pred)       # averages recall over classes
from sklearn.metrics import classification_report
print(classification_report(yte, pred, labels=LABS, digits=3))
# read the per-class recall column before you read accuracy
from sklearn.inspection import permutation_importance
imp = permutation_importance(m, Xte_s, yte, n_repeats=12, random_state=0)
pd.Series(imp.importances_mean, index=cols).sort_values(ascending=False)
```

Session 6 · Causal Inference

```
# -- Load and shape --------------------------------------------------
TRUTH = ess.cat_cafe_truth()      # the planted ATT, cohort by quarter
cells = cells.merge(TRUTH, on=["cohort", "quarter"], how="left")
cells.true_att = cells.true_att.fillna(0.0)
```

```
ok = cells[cells.est.notna()]     # a cell with no clean control is NaN

# -- Fit ------------------------------------------------------------
twfe = smf.ols("dwell_min ~ has_cat + C(cafe_id) + C(quarter)",
               data=d).fit()
# the trap: with staggered timing this is a weighted average that can
# carry NEGATIVE weights, and return the wrong sign at p = 1e-5
import diff_diff as dd
bacon = dd.BaconDecomposition().fit(
    d, outcome="dwell_min", unit="cafe_id", time="quarter",
    first_treat="cat_arrival_quarter")
# shows you exactly which comparisons TWFE made, and with what weight
cs = dd.CallawaySantAnna(control_group="not_yet_treated").fit(
    d, outcome="dwell_min", unit="cafe_id", time="quarter",
    first_treat="cat_arrival_quarter", aggregate="group")
dd.SunAbraham(...)                # interaction-weighted event study
dd.ImputationDiD(...)             # Borusyak-Jaravel-Spiess
dd.TwoStageDiD(...)               # Gardner
dd.HonestDiD(...)                 # Rambachan-Roth sensitivity

# -- Read the fit ---------------------------------------------------
bacon.to_dataframe()[["treated_group", "control_group",
                      "comparison_type", "estimate", "weight"]]
bacon.twfe_estimate, bacon.decomposition_error
cs.overall_att                    # one number, correctly weighted
cs.group_effects[4]["effect"], cs.group_effects[4]["se"]
cs.group_time_effects[(4, 6)]     # one cohort, one quarter

# -- Choose and validate --------------------------------------------
np.average(s.estimate, weights=s.weight)   # a weighted average, by hand
pre = cells[(cells.event_time < 0) & cells.est.notna()]
(pre.est.abs() > 1.96 * pre.se).sum()      # how many pre-periods fire
slope = np.polyfit(pre.event_time, pre.est, 1)[0]
# the pre-trend slope you would have to rule out

# -- Plot -----------------------------------------------------------
ax.errorbar(ev.event_time, ev.est, yerr=1.96 * ev.se,
            fmt="o", ms=4, capsize=2, lw=1.2)
```